

Map Explaining for LCPC

What is map?

`map` is a data structure in C++ STL that stores data as **key-value pairs**.

Each element has:

- **Key**: the identifier used to access the data.
- **Value**: the information associated with that key.

General form:

```
map<key_type, value_type> name;
```

Example:

```
map<string, int> age;
```

Here:

- Key = `string`
- Value = `int`

It stores relationships like:

```
Ali -> 20  
Sara -> 18
```

Creating a map

```
map<int, int> mp;
```

This creates an empty map where:

- Key is an integer.
 - Value is an integer.
-

Adding elements

We can add elements using:

```
mp[key] = value;
```

Example:

```
mp[5] = 100;  
mp[10] = 50;
```

The map becomes:

```
5 -> 100  
10 -> 50
```

If we assign a value to an existing key:

```
mp[5] = 200;
```

The old value is replaced:

```
5 -> 200
```

Accessing values

To get a value:

```
cout << mp[5];
```

Output:

```
200
```

Counting frequencies using map

One of the most common uses of `map` in competitive programming is counting occurrences.

Example:

Array:

```
5 7 5 8 5 7
```

Code:

```
map<int,int> freq;  
  
for(int x : arr)  
{  
    freq[x]++;  
}
```

Result:

```
5 -> 3  
7 -> 2  
8 -> 1
```

Explanation:

Every time we see a number:

```
freq[x]++;
```

we increase its counter by one.

If the key does not exist, `map` creates it automatically with value `0`.

Getting the number of distinct elements

`map.size()` returns the number of keys.

Example:

```
5 -> 3
7 -> 2
8 -> 1
```

The size is:

```
freq.size()
```

Output:

```
3
```

This means there are 3 different numbers.

Iterating through a map

Each element in a map is a `pair`.

Example:

```
for(auto x : mp)
{
    cout << x.first << " " << x.second << endl;
}
```

`x.first` :

- The key.

`x.second` :

- The value.

Example:

Map:

```
Ali -> 20  
Sara -> 18
```

Output:

```
Ali 20  
Sara 18
```

Checking if a key exists

Use:

```
mp.count(key)
```

Example:

```
if(mp.count(5))  
{  
    cout << "Found";  
}  
else  
{  
    cout << "Not Found";  
}
```

`count()` returns:

- `1` if the key exists.
- `0` if the key does not exist.

Important warning

Do not use:

```
if(mp[key])
```

to check existence.

Because if the key does not exist:

```
mp[key]
```

will create it automatically with value `0`.

Use:

```
mp.count(key)
```

instead.

Deleting an element

Remove a key using:

```
mp.erase(key);
```

Example:

```
mp.erase(5);
```

The key `5` is removed.

Time Complexity

For `map`:

- Insert: `O(log n)`
- Search: `O(log n)`
- Delete: `O(log n)`

Because `map` is usually implemented using a balanced tree.

Common Competitive Programming Patterns

1. Frequency counting

```
map<int,int> freq;

for(int x : v)
    freq[x]++;
```

Used for:

- Counting numbers.
 - Counting characters.
 - Finding most frequent elements.
-

2. Finding maximum frequency

Example:

```
string ans;
int mx = 0;

for(auto x : freq)
{
    if(x.second > mx)
    {
        mx = x.second;
        ans = x.first;
    }
}
```

Used in problems like:

- Finding the team with the most wins.
 - Finding the most repeated element.
-

Examples of problems using map

- Codeforces 43A - Football
- Codeforces 4C - Registration System

Summary

Think about `map` as a dictionary:

```
Key ----> Value
```

In competitive programming:

- Key is usually the element we want to track.
- Value is usually a counter or information related to that element.

The most important line to remember:

```
mp[x]++;
```

It means:

"Increase the count of x by one."

`map` is one of the most useful STL containers for solving Codeforces and LCPC problems.